

软件工程 II 期末复习手册

依据：语音版重点、考试题目类型与重点、123.txt 答疑转写、今年课件。复习提纲.pdf 仅作参考。

组织方案：我采用“按考试链路综合总结”，而不是逐章机械摘录。理由是今年大题明确基于同一个系统展开，按“需求 → 体系结构 → 详细设计 → 构造 → 测试 → 人机交互”的链路复习，更接近实际答题过程。每个环节同时保留对应课件截图，方便按课程标准作答。

题型定位

- 简答题：1 道，含 3 小题。重点准备概念辨析、设计原则、UML 图规则、PO/VO、状态图主体等短答。
- 大题：9 道，围绕同一个系统连续展开，覆盖需求、体系结构设计、详细设计、构造、测试、人机交互。
- 作答策略：先读系统背景，统一业务目标和核心用例；后续每道题都沿用同一个系统语境，避免前后不一致。

今年复习权重

层级	高优先级	低优先级/不考倾向
需求	业务/用户/系统级需求；用例、场景、系统顺序图、概念类图；功能/非功能需求	需求规格说明书完整格式不会要求手写
体系结构	分层、MVC、包图、4+1 视图、接口、PO/VO、DIP	包的原则今年不考；技术细节不考
详细设计	GRASP、控制器、信息专家、Creator、控制风格、SOLID/类设计原则、状态图主体	单纯背设计模式名字不是重点
构造/测试/HCI	构造质量、重构、基本测试层次与方法、人机交互原则和评价	测试相关图不重点；管理团队概念不重点

一、总览：一套系统题怎么展开

遇到基于同一个系统的大题，可以把答案想成一条可追溯链：业务目标决定需求，需求决定架构，架构约束详细设计，详细设计落到构造与测试，人机交互则从用户使用质量上反过来检验需求。

阶段	输入	关键产物	考试高频动作
需求	业务痛点、用户任务、约束	业务需求、用户需求、系统级需求、用例、概念类图、系统顺序图	区分层次；写用例正常/扩展流程；画 Actor-系统交互
体系结构	功能域、非功能需求、项目约束	分层/MVC、包图、接口、PO/VO、4+1 视图	说明为什么选分层；画包和依赖；标接口和数据对象
详细设计	用例、概念类图、系统顺序图、架构接口	设计类图、详细顺序图、状态图、职责分配	按 GRASP 分配职责；分析 SOLID；区分关联/聚合/组合
构造	详细设计和接口规格	可读代码、异常/断言、防御式编程、重构	说明代码质量和构造活动，不纠缠框架注解
测试	需求、代码、状态/流程	单元/集成/系统测试；黑盒/白盒用例	写测试用例思路；图相关不作为重点
人机交互	用户、任务、场景、界面原型	交互过程、原型、可用性原则、评估	用原则解释界面好坏，理由充分即可

核心提醒：AI 可以生成初稿，但你必须判断它的层次、边界、关系、交互顺序和设计原则是否正确。这一点在需求、UML 图、架构和设计原则题里都可能体现。

二、需求工程：今年最应吃透的起点

1. 需求三层：业务需求 → 用户需求 → 系统级需求

需求不是一句“系统要做什么”。课程强调需求的层次性：业务需求回答为什么建系统，用户需求回答用户要借系统完成什么任务，系统级需求把用户任务规格化为系统与外部实体的交互行为。

三个层次的需求示例 [SWEBOK 1.2]

业务需求 (Business Requirements)

R2: 在系统使用 3 个月后，销售额度应该提高 20%。

用户需求 (User Requirements)

R3: 系统要帮助收银员完成销售处理。

系统级需求 (System Requirements)

R4: 收银员输入购买的商品时，系统要显示该商品的描述、单价、数量和总价。

💡 从 R2 到 R4，需求越来越具体、可测试、可实现。
但同时也越来越远离业务目标——三个层次缺一不可。

22

课件截图：02-01-需求工程基础.pdf 第 22 页，三个层次的需求示例。

1. 业务需求：从组织/用户业务指标出发，要求可度量，例如提高销售额、降低结账时间、降低错误率。它定义系统目标、范围和共同前景。
2. 用户需求：从用户角度描述系统支持的任务，常用用例、场景、用户故事表达。注意不要混入技术实现和界面细节。
3. 系统级需求：从系统角度精确描述对外交互、输入输出、状态变化和约束，是开发和验收的依据。

答题模板：先写“业务目标/痛点”，再写“用户通过系统完成的任务”，最后写“系统响应、数据、约束和可验证条件”。不要直接从业务目标跳到数据库/接口实现。

2. 功能需求与非功能需求

功能需求描述系统做什么；非功能需求描述系统如何好地做，例如可靠性、可用性、安全性、可维护性、可移植性、易用性等。答架构题时，非功能需求经常是选择架构风格和模块边界的理由。

常见质量属性与案例 [SWEBOK 1.7]

质量属性	定义	超市系统案例
可靠性 (Reliability)	规格条件下执行所要求能力	QA1: 网络故障时, 系统不能出现故障
可用性 (Availability)	可操作和可访问的概率	QA2: 系统可用性达到98%
安全性 (Security)	阻止未授权访问	QA3: VIP顾客只能查看自己的信息
可维护性 (Maintainability)	修改以排障/改进的容易程度	QA4: 新增特价类型在2人月内完成
可移植性 (Portability)	跨环境转换能力	QA5: 1人月内从Windows换到Solaris
易用性 (Usability)	用户使用的努力和满意度	QA6: 使用1月的收银员效率达10件/分钟

💡 [Robert1990]: 在决定系统成功或失败的因素中, 满足非功能属性需求往往比满足功能性需求更为重要。

32

课件截图：02-01-需求工程基础.pdf 第 32 页，常见质量属性与案例。

3. 用例：不是 CRUD 列表，而是用户目标

用例应对应参与者的一次完整业务目标，完成后用户有“任务达成”的感受。常见错误是把“登录、增加、删除、查询数据库”等技术或步骤当成用例。

用例的粒度判断 —— Cockburn 三层次 [Cockburn 2001]

层次	别名	说明	示例
概要层	☁ 云层	涉及多个用户目标的长期过程	"管理书店运营"——粒度过粗
用户目标层	🌊 海平面	参与者的一个完整业务目标	"购买图书""搜索图书"—— 最佳粒度
子功能层	🌊 水下	实现目标的一个步骤	"输入密码""验证信用卡"——粒度过细

判断经验法则

- 该用例能否在一次会话中完成？
- 完成后用户是否有一种**"任务达成"的满足感**？
- 如果两个都是"是" → 用户目标层 ✅

28

课件截图：02-03-用例建模与用户故事.pdf 第 28 页，Cockburn 三层次粒度判断。

项目	答题要点
用例图	包含参与者、用例、关系、系统边界。include 表示必然复用，extend 表示可选/条件扩展。

项目	答题要点
用例描述	ID、名称、参与者、触发条件、前置条件、后置条件、正常流程、扩展流程、特殊需求、补充说明。
场景	一条具体路径。正常流程是主成功场景；扩展流程是异常、分支、替代路径。
用户故事	角色 + 功能 + 价值，注意 INVEST：独立、可讨论、有价值、可估计、小、可测试。

用例描述模板（10 个字段）—— 基本信息 [SWEBOK 4.2]

字段	内容描述
ID	用例的唯一标识（如 UC1、UC2）
名称	对用例内容的精确描述
参与者	描述系统的参与者和每个参与者的目标
触发条件	标识启动用例的事件
前置条件	用例能够正常启动和工作的系统状态条件
后置条件	用例执行完成后的系统状态条件

33

课件截图：02-03-用例建模与用户故事.pdf 第 33 页，用例描述模板基本信息。

用例描述模板（10 个字段）—— 行为与补充 [SWEBOK 4.2]

字段	内容描述
正常流程	在常见和符合预期的条件下，系统与外界的行为交互序列
扩展流程	用例中可能发生的其他场景（异常、分支）
特殊需求	和用例相关的其他特殊需求，尤其是非功能性需求
补充说明	其他需要记录的信息

34

前 6 个字段描述上下文，后 4 个字段描述行为与约束

课件截图：02-03-用例建模与用户故事.pdf 第 34 页，用例描述模板行为与补充。

正常流程 vs 扩展流程 [SWEBOK 4.2]

正常流程 (Main Success Scenario)

1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 10 → 11 → 12

2a: 非法ID 3a: 非法商品 6a: 无特价策略
3b: 多件 3-7a: 删除商品
3-7b: 取消交易

- 每一条扩展都是一个独立的场景 (Scenario)
- 所有场景的集合 = 用例"销售处理"

💡 写扩展流程时, 要考虑**每一步可能出错的情况**——这是人工审查的重点

39

课件截图：02-03-用例建模与用户故事.pdf 第 39 页，正常流程 vs 扩展流程。

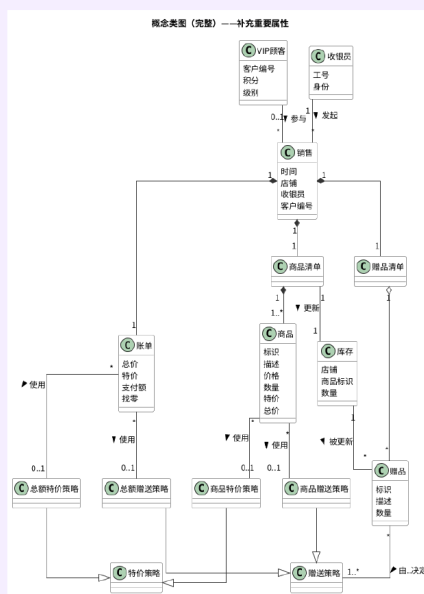
4. 概念类图：分析阶段的静态模型

概念类图描述问题域中的概念、属性、关联和多重性。答疑明确：概念类图只有类和属性，不含方法。不要把实现类、数据库表操作、接口方法混进去。

概念类图示例（完整）[SWEBOK 4.4]

补充重要属性

- 销售：时间、店铺、收银员、客户编号
- 商品：标识、描述、价格、数量、特价、总价
- 账单：总价、特价、支付额、找零
- VIP顾客：客户编号、积分、级别
- 库存：店铺、商品标识、数量
- 收银员：工号、身份
- 赠品：标识、描述、数量
- 赠送策略 ← 商品赠送 / 总额赠送
- 特价策略 ← 商品特价 / 总额特价



22

课件截图：02-04-需求分析与AI辅助建模.pdf 第 22 页，概念类图完整示例。

- 识别方法：从用例文本中抽取名词和名词短语，去重、合并同义词，再用“是否有状态/行为、是否需被系统记录”筛选。

- 关联关系：普通关联表示长期持有引用；聚合有整体-部分但生命周期不绑定；组合有生命周期绑定，整体消亡则部分消亡。
- 检查：是否覆盖关键用例？是否把纯属性误建为类？多重性是否有业务规则支撑？

5. 系统顺序图：从行为角度建模系统级需求

顺序图是从行为/交互角度建模。需求阶段重点是系统顺序图：参与者与系统之间的消息顺序，把系统看成黑盒。详细设计阶段的顺序图则展开系统内部对象/类之间的协作。

从用例描述到顺序图 [SWEBOK 4.4]

映射规则

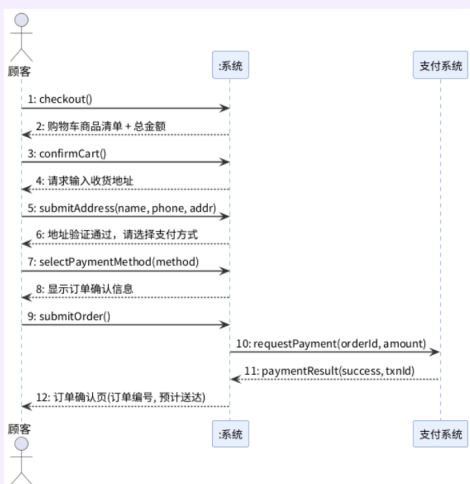
用例描述中的内容	顺序图中的对应
参与者的动作	从参与者到系统的消息
系统的响应	从系统到参与者的返回消息
每一步交互	一条消息箭头
交互的时间顺序	从上到下的消息排列

用例描述中的每一步交互 → 顺序图中的一条消息。
正常流程先画，扩展流程用 `alt` / `opt` 片段补充。

28

课件截图：02-04-需求分析与AI辅助建模.pdf 第 28 页，从用例描述到顺序图的映射。

案例 —— “购买图书”系统顺序图 [SWEBOK 4.4]



三方参与者

顾客 ↔ 系统 ↔ 支付系统

系统操作

- `checkout()` — 请求结算
- `confirmCart()` — 确认购物车
- `submitAddress(...)` — 提交地址
- `selectPaymentMethod()` — 选支付方式
- `submitOrder()` — 提交订单

外部系统交互

- `requestPayment()` → 支付系统
- `paymentResult()` ← 支付系统

30

课件截图：02-04-需求分析与AI辅助建模.pdf 第 30 页，“购买图书”系统顺序图。

系统顺序图 vs 详细顺序图：系统顺序图画 Actor -> System 的系统操作；详细顺序图画 Controller、Service、Entity、DAO 等对象之间如何协作。答题时先判断题目处于需求分析还是详细设计。同步消息是实线 + 实心三角箭头；异步消息是虚线 + 鱼骨箭头。

6. 状态图：先明确主体

答疑反复强调：画状态图之前必须明确主体。主体可以是订单、图书、POS、单个红绿灯，也可以是整个路口；主体不同，状态集合和转换事件完全不同。

状态图概念 [SWEBOK 4.4]

- 描述系统、用例或对象在**整个生命期内的状态变化**
- 适用场景：订单状态流转、设备状态、用户账户状态等
- 也称为"状态机图" (State Machine Diagram)

基本元素

元素	符号	说明
状态 (State)	圆角矩形	对象在某一时刻的条件或情况
转换 (Transition)	箭头	从一个状态到另一个状态的变迁
事件 (Event)	转换上的标签	触发状态转换的外部刺激
动作 (Action)	/ action	状态转换时执行的操作
初始状态	实心圆	状态机的起点
终止状态	圆中圆	状态机的终点

38

课件截图：02-04-需求分析与AI辅助建模.pdf 第 38 页，状态图概念。

状态图 → Java 代码的映射 [SWEBOK 4.4]

映射规则

状态图元素	Java 对应
状态	枚举值
转换	方法
守卫条件	if 检查
非法转换	抛出异常
动作	方法内逻辑

状态图中的每个转换 → 一个方法；每个守卫条件 → 方法开头的 if 检查

代码示例

```
public enum OrderStatus {
    PENDING_PAYMENT, PAID, SHIPPED,
    COMPLETED, CANCELLED, REFUNDING
}

// 待支付 → 已支付
public void pay() {
    if (status != PENDING_PAYMENT)
        throw new IllegalStateException();
    status = PAID;
    deductInventory(); // 动作：扣减库存
}

// 已支付 → 已发货
public void ship(String trackingNo) {
    if (status != PAID)
        throw new IllegalStateException();
    status = SHIPPED;
}

// 多条 → 已取消 (守卫条件不同)
public void cancel() {
    if (status == PENDING_PAYMENT) {
        status = CANCELLED;
        releaseInventory(); // 释放库存
    } else if (status == PAID) {
        status = CANCELLED;
        releaseInventory();
        initiateRefund(); // 发起退款
    } else throw new IllegalStateException();
}
```

43

课件截图：02-04-需求分析与AI辅助建模.pdf 第 43 页，状态图到 Java 代码的映射。

三、体系结构设计：重点是分层、MVC、包图和接口

1. 架构风格怎么选

体系结构设计不是凭喜好选技术，而是用功能需求、非功能需求和项目约束驱动。课程和答疑都强调：大多数企业/管理类系统优先考虑分层架构，必要时结合 MVC。

步骤 2：选择体系结构风格

需求特征	推荐风格	原因
企业级业务系统、团队分工	分层	职责分明、易于分工
数据流式处理、转换链	管道-过滤器	数据逐步变换
高交互、多视图同步	MVC	界面与逻辑分离
大规模、独立部署、多团队	微服务	独立开发/部署/扩展
松耦合、异步通信	事件驱动	发布/订阅解耦
小型工具、脚本	主程序-子程序	简单直接

实践建议：大多数企业系统首选分层架构，在此基础上根据需要引入其他风格。

40

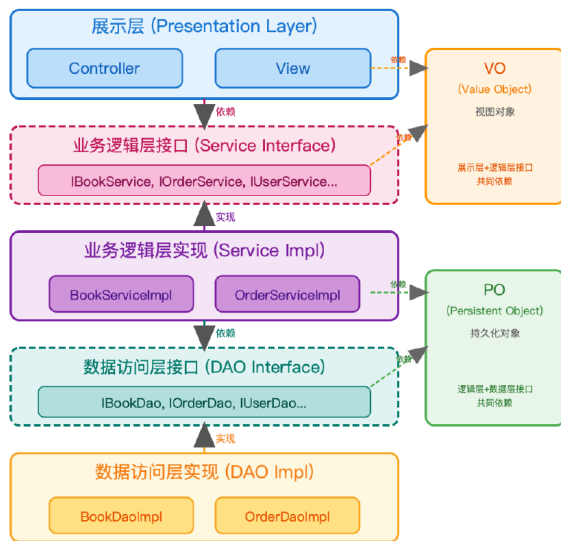
课件截图：03-02-体系结构风格与设计过程.pdf 第 40 页，按需求特征选择体系结构风格。

- 主程序-子程序：简单直接，适合小型工具。
- 面向对象：对象封装数据和行为，通过消息协作，是分层和 MVC 的基础。
- 分层：展示层、业务逻辑层、数据访问层；职责清晰，便于维护和并行开发。
- MVC：Model 管理业务数据与逻辑，View 展示，Controller 处理输入并协调。实践中常嵌入展示层。

2. 包图怎么画

答疑中老师建议包图至少画出展示层、逻辑层、数据层、层间接口，以及 PO、VO。包图用于表现体系结构的逻辑视图，评分看整体结构，但能画出包内依赖更好。

初步设计方案三 — 接口独立层与依赖倒置



核心改进

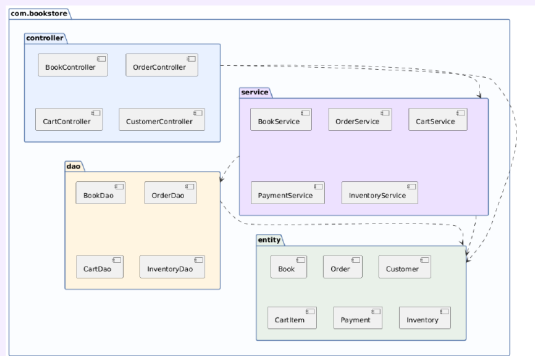
改进点	说明
接口独立成层	Service Interface 和 DAO Interface 作为独立层
实现继承接口	Service Impl → implements → Service Interface
上层依赖接口	展示层依赖 Service Interface，不依赖实现
VO/PO分离	VO用于展示层-逻辑层，PO用于逻辑层-数据层

依赖关系

依赖方向	说明
展示层 → Service Interface	上层依赖接口
Service Impl → Service Interface	实现继承接口
Service Impl → DAO Interface	上层依赖接口
DAO Impl → DAO Interface	实现继承接口
展示层 + Service Interface → VO	共同依赖值对象
Service Impl + DAO Interface → PO	共同依赖持久化对象

课件截图：03-02-体系结构风格与设计过程.pdf 第 51 页，接口独立层、依赖倒置与 PO/VO 分离。

案例：在线书店 — 逻辑视图（包图）



每个顶层包内部按业务模块再次划分（book / order / cart / customer）。

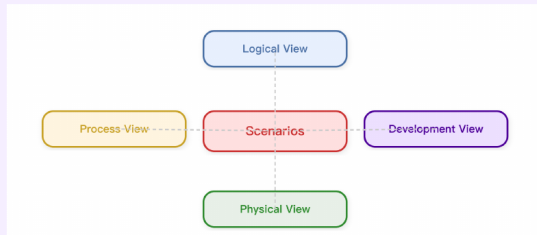
课件截图：03-03-体系结构设计实践与验证.pdf 第 29 页，在线书店逻辑视图包图。

包图答法：先按层画大包，再按功能域拆包；依赖方向通常从展示层到接口、业务实现到 DAO 接口，具体实现依赖接口。层间简单、层内可复杂。小错误通常看整体，但接口、PO/VO、依赖方向要尽量清楚。

3. 4+1 视图

4+1 视图用于从多个角度理解体系结构。考试若问体系结构视图，重点记逻辑视图可用包图/类图表达，场景视图用关键用例穿透验证其他视图。

Kruchten 4+1 视图模型



每个视图面向不同的利益相关者，解决不同的关注点。

27

课件截图：03-03-体系结构设计实践与验证.pdf 第 27 页，Kruchten 4+1 视图模型。

视图	关注点	常用图
逻辑视图	功能划分、类与包组织	包图、类图
开发视图	代码组织、模块划分	组件图
进程视图	并发、同步、性能	活动图、序列图
物理视图	部署拓扑、硬件映射	部署图
场景视图	用例驱动，验证其他视图	用例图

4. PO、VO、供接口与需接口

PO 与 VO 概念

概念	全称	用途	所在层
PO	Persistent Object	与数据库表一一对应	entity / dao
VO	Value Object	面向展示层的数据传输对象	controller / service

为什么需要区分？

- PO 包含数据库细节（主键、外键、时间戳）
- VO 只包含前端需要的字段
- 隔离变化：数据库字段变更不影响前端接口

```
// PO: 对应 book 表
public class Book { Long id; String isbn; String title; BigDecimal price; ... }
// VO: 前端展示用
public class BookVO { String isbn; String title; String priceText; double rating; }
```

39

课件截图：03-03-体系结构设计实践与验证.pdf 第 39 页，PO 与 VO 概念。

- PO：Persistent Object，持久化对象，服务于数据层与逻辑层交互，常对应数据库/ORM。
- VO：Value Object，展示层与逻辑层交互的数据对象，只返回前端需要的数据，避免传递多余信息。
- 供接口：模块对外提供什么能力。需接口：模块为了完成职责需要调用别人什么能力。二者共同构成完整的体系结构接口。

四、详细设计：今年核心中的核心

1. 详细设计的输入与输出

详细设计是把需求和体系结构递归精化到类、方法、对象协作和算法层面。输入包括用例、概念类图、系统顺序图、状态图、非功能需求，以及架构中的模块规格和接口。输出包括设计类图、详细顺序图、状态图、伪代码/流程图、设计理由。

2. GRASP：职责分配的框架

GRASP Patterns

- **GRASP** = **G**eneral **R**esponsibility **A**ssignment **S**oftware **P**atterns
- Not 'design patterns', rather **fundamental principles** of object design
- Focus on one of the most important aspects of object design: **assigning responsibilities to classes**

5 个核心模式（本讲覆盖）

#	模式	解决的问题
1	Low Coupling	降低模块之间的耦合
2	High Cohesion	保持模块职责的聚焦
3	Information Expert	把职责分配给"拥有信息"的类
4	Creator	由谁负责创建对象
5	Controller	由谁负责处理系统事件

Larman 著《UML 和模式应用》是 OO 详设的经典参考。

 SWEBOOK v4 §5.4 · Object-Oriented Design | §1.4 · Software Design Principles (Coupling, Cohesion)

25

课件截图：04-01-详细设计.pdf 第 25 页，GRASP 五个核心模式。

GRASP	考试答法
Low Coupling	分配职责时减少依赖，优先依赖接口，避免一个类变化牵动多个类。
High Cohesion	一个类承担紧密相关职责，避免万能类。
Information Expert	谁拥有完成职责所需信息，谁承担职责；常体现为委托链。
Creator	由包含、聚合、记录、密切使用或拥有初始化数据的类创建对象。
Controller	由外观控制器、系统控制器、角色控制器或用例控制器处理系统事件。

③ Information Expert (信息专家)

- **Problem:** What is the most basic principle by which responsibilities are assigned in object-oriented design?
- **Solution:** Assign a responsibility to the class that has the information necessary to fulfill the responsibility.

案例

要计算 `Sales.total()` :

- 谁知道每个 `SalesLineItem` 的数量? → `SalesLineItem`
- 谁知道每个 `Commodity` 的价格? → `Commodity`
- → `Sales.total()` 委托给 `SalesLineItem.subtotal()` , 后者再调 `Commodity.getPrice()`

"谁有数据, 谁负责" —— 自然形成职责链, 避免数据外泄。

28

课件截图 : 04-01-详细设计.pdf 第 28 页 , Information Expert 。

⑤ Controller (控制器)

- **Problem:** How to assign responsibility for handling a system event?
- **Solution:** If a program receives events from external sources other than its graphical interface, add an event class to **decouple the event source(s) from the objects that actually handle the events.**

4 种可选的控制器

1. **The business or overall organization** (façade controller) — e.g. `Store`
2. **The overall "system"** (façade controller) — e.g. `POST`
3. **An animate thing in the domain** that would perform the work (role controller) — e.g. `Cashier`
4. **An artificial class** (Pure Fabrication) representing the use case (use case controller) — e.g. `BuyItemsHandler`

 SWEBOOK v4 §5.4 · Object-Oriented Design | §3.2 · Control and Event Handling

31

课件截图 : 04-01-详细设计.pdf 第 31 页 , Controller 。

3. 控制风格

Collaboration (Control) Styles

A control style is a way that all system behavior is distributed among objects (components) network.

风格	特征	控制复杂度
Centralized (集中式)	A few controllers record logics of all system behaviors	控制器高复杂度，其他对象低复杂度
Delegated (委托式)	Decision making is distributed through the object networks with a few controllers making the main decisions	中复杂度对象有几个，其他低复杂度
Dispersed (分散式)	All system behavior is spread widely through the objects network	所有对象都低复杂度，但交互密集

34

课件截图：04-01-详细设计.pdf 第 34 页，集中式/委托式/分散式控制风格。

- 集中式：少数控制器记录大部分逻辑，容易找决策点，但控制器易膨胀。
- 委托式：控制器保留主决策，把子任务交给领域对象，通常是实际系统的折中方案。
- 分散式：行为散布到大量对象中，控制流难追踪，通常谨慎使用。

4. 关联、聚合、组合

关系	含义	判断点
普通关联	A 长期持有 B 的引用	没有特殊整体-部分含义
聚合	整体与部分关系，但生命周期不绑定	部分可脱离整体存在或被共享
组合	强整体-部分关系，生命周期绑定	整体消亡，部分也消亡

5. 类设计原则/SOLID：很可能以简答或代码分析出现

原则汇总（P1—P7）：信息隐藏基础 · 访问耦合 · 继承耦合

完整编号体系：P1—P4 见 04-01（信息隐藏基础），P5—P14 见本讲（面向对象设计）

编号	缩写	原则	核心思想	SWEBOK
P1	—	全局变量有害	全局变量穿透所有模块，是隐藏的反义词	§1.4 Encapsulation
P2	—	优先显示表达	用接口/文档显式表达约束，不靠隐式约定	§1.4
P3	DRY	不要重复	同一决策不在多处编码；重复=多处修改	§1.4
P4	—	面向接口编程	依赖接口而非实现，降低访问耦合	§1.4 Sep. of I&I; §5.4
P5	LoD	迪米特法则	只与直接朋友交谈，避免消息链	§5.4 OOD
P6	ISP	接口隔离	接口细化，客户端只依赖需要的方法	§5.4 SOLID-I
P7	LSP	里氏替换	子类前置放宽、后置加强、不变式维持	§5.4 SOLID-L

37

课件截图：04-02-面向对象的模块化与信息隐藏.pdf 第 37 页，原则 P1-P7。

原则汇总（P8—P14）：封装变更 · SOLID · SOFA

编号	缩写	原则	核心思想	SWEBOK
P8	—	组合优于继承	黑盒复用，避免继承耦合	§5.4 GoF
P9	—	为变更而封装	隔离可变部分，稳定不变部分	§1.4 Encapsulation
P10	—	权限最小化	从 <code>private</code> 开始，按需放宽	§1.4
P11	OCP	开闭原则	对扩展开放，对修改关闭	§5.4 SOLID-O
P12	DIP	依赖倒置	高层依赖抽象，不依赖具体实现	§5.4 SOLID-D
P13	SRP	单一职责	一个类只有一个变化原因	§5.4 SOLID-S
P14	SOFA	方法设计	Short · One thing · Few args · Abstraction	§5.4 SOFA

记忆结构：P1—P4 防止"隐式依赖"；P5—P8 控制"耦合来源"；P9—P13 管理"变化封装"；P14 约束"方法粒度"。

 SWEBOK v4 §1.4 · Software Design Principles | §5.4 · Object-Oriented Design (SOLID)

38

课件截图：04-02-面向对象的模块化与信息隐藏.pdf 第 38 页，原则 P8-P14。

原则	一句话	常见反例
SRP	一个类只有一个变化原因	Employee 同时算薪资、存数据库、生成报表
OCP	对扩展开放，对修改关闭	每加一种类型都改 if/else 或 switch
LSP	子类可替换父类而不破坏正确性	Penguin 继承 Bird 但 fly() 抛异常
ISP	客户端不依赖不用的方法	Robot 被迫实现 eat/sleep
DIP	高层和低层都依赖抽象	BusinessLogic 直接依赖 MySQLDatabase

原则	一句话	常见反例
LoD	只与你的直接朋友交谈	customer.getAddress().getCity() 消息链
SOFA	方法短、只做一件事、少参数、抽象层次一致	方法又校验订单又直接写 SQL

答疑补充：今年内聚/耦合不是最重点，但控制耦合和印记耦合要会识别。控制耦合是传 flag 让被调用方法内部 switch/if 决定行为，可用多态/策略改进。印记耦合是传入超出方法所需的数据对象，应只传必要参数或抽象接口。

6. 设计模式：重在原则诱导，不是背名字

今年设计模式题会通过小问题诱导你根据设计原则完成设计，不一定要写出模式名称。重点思路是识别变化点，使用接口、组合、多态、依赖倒置来隔离变化。

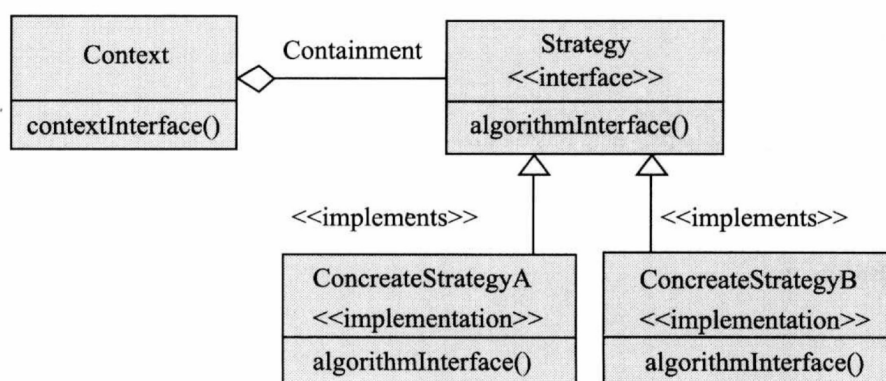


图 16-8 策略模式

策略模式：定义了算法族，分别封装起来，让他们之间可以互相替换，此模式让算法的变化独立于使用算法的客户。

策略模式类图

课件截图：设计模式.pdf 第 35 页，策略模式类图。

- 策略模式：把算法族封装为统一接口，使算法变化独立于使用算法的客户；可消除根据类型分支的 switch。
- 工厂方法/抽象工厂：用于封装对象创建变化。注意标准 GoF 中没有笼统的“工厂模式”一个名字。
- 迭代器：隐藏集合内部结构，只暴露遍历能力。
- 单例：确保一个类只有一个实例，注意全局状态和测试困难。

五、构造：从设计落到可维护代码

构造不是单纯写代码，而是编码、验证、单元测试、集成、调试、构建管理和重构的综合活动。老师强调不会考 Java/Spring 注解等具体技术细节，但基本概念、伪代码、断言、异常、重构、代码质量要会。

重构

- 为什么要重构？
 - 1、因为无法预计到后续数年的修改，导致软件开发阶段的设计方案不能满足修改要求；
 - 2、随着修改次数的增多，软件设计结构的质量越来越脆弱，很难继续维持可修改性。
- 什么是重构？
 - 修改软件系统的严谨方法，它在不改变代码外部表现的情况下改进其内部结构。
- 重构的时机？
 - 增加新的功能时。需要注意的是重构发生在新功能增加完成之后，用来消除新功能所添加代码导致的坏味道；而不是发生在新功能添加之前，重构不改变代码外部行为，不是能够实现新功能添加的方法。
 - 发现了缺陷进行修复时。诊断缺陷时如果发现代码存在坏味道或者修复代码会引入坏味道，就需要进行重构。
 - 进行代码评审时。如果在评审代码时发现了坏味道，就需要进行重构。

课件截图：软件构造.pdf 第 26 页，重构。

代码的坏味道

- **太长的方法**，往往意味着方法完成了太多的任务，不是功能内聚的，需要被分解为多个方法。[McConnell2004]认为如果方法代码长度超过了一个屏幕，就需要留心注意了。
- **太大的类**，往往意味着类不是单一职责的，需要被分解为多个类。
- **太多的方法参数**，往往意味着方法的任务太多或者参数的数据类型抽象层次太低，不符合接口最小化的低耦合原则，需要将其分解为多个参数少的方法或者将参数包装成对象、结构体等抽象层次更高的数据类型。
- **多处相似的复杂控制结构**，例如多处相同类型的Case结构，往往意味着多态策略不足，需要使用继承树多态机制消除复杂控制结构。
- **重复的代码**，往往意味着隐式耦合，需要将重复代码提取为独立方法。一个类过多使用其他类的属性，往往意味着属性分配不正确或者协作设计不正确，需要在类间转移属性或者使用方法委托代替属性访问。
- **过多的注释**，往往意味着代码的逻辑结构不清晰或者可读性不好，需要进行逻辑结构重组或者代码重组。

课件截图：软件构造.pdf 第 27 页，坏味道与重构线索。

- 重构目的：在不改变外部可观察行为的前提下改善内部结构。常在新增功能后、修复缺陷后、代码评审发现坏味道时进行。
- 常见坏味道：过长方法、过大的类、重复代码、魔法数字、多处相似复杂控制结构、过多注释掩盖结构不清。
- 防御式编程：对外部输入、外部依赖、异常状态进行校验；异常用于可恢复或外部错误，断言用于检查程序内部不该违反的条件。
- 圈复杂度：控制流越复杂，测试和理解成本越高。复杂逻辑要拆分、表驱动或用多态/策略消除分支。

构造题答法：先说这段代码/设计暴露了什么质量问题，再说明违反哪个原则，最后给出可落地的重构方式，例如提取方法、拆分类、引入接口、使用策略、多态替代 flag。

六、测试：方法要会，图相关不作为重点

答疑中关于测试图的回复是“图相关的不需要考虑吧”。因此测试复习重点放在测试层次、测试用例选择、黑盒/白盒方法、桩与驱动、基于需求/代码设计测试，而不是专门准备测试相关图。

1. 测试层次

系统测试

- 单元测试、集成测试更加关注技术上的正确性，重在发现设计缺陷和代码缺陷。系统测试则不同，它更关注不符合需求的缺陷和需求自身的内在缺陷。
- 根据测试目标的不同，有很多不同类型的系统测试：功能测试、非功能性测试、验收测试、安装测试等等。但是发生在软件测试阶段，完全由软件测试人员控制和执行的主要是功能测试和非功能性测试。
- 系统测试关注整个系统的行为，所以不依赖于桩程序和驱动程序。但是，使用一些测试工具可以让系统测试过程更加自动化。
- 系统测试的功能测试计划以需求规格说明文档或用例文档为基础，主要使用随机测试和基于规格的测试技术设计功能测试用例。在测试非功能性需求时需要使用针对非功能需求的特定测试技术进行测试计划和测试用例设计。

课件截图：软件测试.pdf 第 31 页，系统测试。

层次	关注点	常见依据
单元测试	类/方法级正确性，常需驱动和桩	方法规格、代码逻辑、边界条件
集成测试	类/模块之间协作是否正确	接口、调用关系、桩与驱动
系统测试	整个系统是否符合需求	需求规格、用例文档、业务场景
回归测试	修改后原有行为是否保持	历史缺陷、变更影响范围

2. 黑盒测试

黑盒测试方法

- 等价类划分

有效等价类：是指对于程序的规格说明来合理、意义输入数据构成集合。利用有效等价类可检验程序是否实现了规格说明中所定的功能和性能。

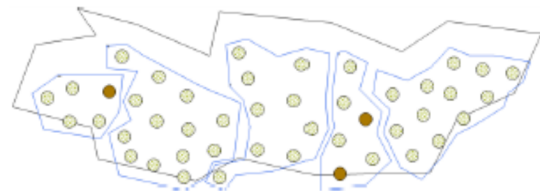


图 19-8 等价类划分

课件截图：软件测试.pdf 第 37 页，等价类划分。

黑盒测试方法

- 边界值分析

表 19-4 Sales 类 getChange(double payment)方法的边界值分析测试用例

ID	输入		预期输出
1	payment=50	total=50	0
2	payment=50	total=49	1
3	payment=50	total=51	输入数据无效
4	payment=1	total=1	0
5	payment=0	total=0	0
6	payment=-1	total=10	输入数据无效

课件截图：软件测试.pdf 第 40 页，边界值分析。

- 等价类划分：把输入划分为有效/无效等价类，每类选代表值。
- 边界值分析：错误常发生在边界，选最小值、最小值+1、最大值-1、最大值以及非法边界。
- 决策表：多条件组合时列出条件、动作和规则。
- 状态转换测试：若对象生命周期复杂，可覆盖合法转换和非法转换，但今年图相关不作为重点。

3. 白盒测试

白盒测试方法

- 语句覆盖

语句覆盖设计测试用例的标准是确保被测试对象的每一行程序代码都至少执行一次。

```
public Class Customer {  
    int bonus;//积分额度  
    .....  
    //预计算消费行为后的积分额  
    int getBonus ( boolean cashPayment , int consumption , bo  
        //已有的 bonus 需要调整 getBonus，不能直接使用  
        int preBonus=this.getBonus();  
        if (cashPayment) {  
            preBonus+=consumption;  
        }  
        if (vip) {  
            preBonus*=1.5;  
        } else {  
            preBonus*=1.2;  
        }  
        return preBonus;  
    }  
}
```

课件截图：软件测试.pdf 第 47 页，语句覆盖。

白盒测试方法

- 路径覆盖

路径覆盖测试用例的标准是确保程序中每条独立执行路径都至少执行一次

表 19-8 Customer.getBonus() 的路径覆盖测试用例

ID	输入				预期输出
1	getBonus ()=100	cashPayment=true	consumption=100	vip=true	300
2	getBonus ()=100	cashPayment=false	consumption=100	vip=false	120
	getBonus ()=100	cashPayment=true	consumption=100	vip=false	240
	getBonus ()=100	cashPayment=false	consumption=100	vip=true	150

课件截图：软件测试.pdf 第 52 页，路径覆盖。

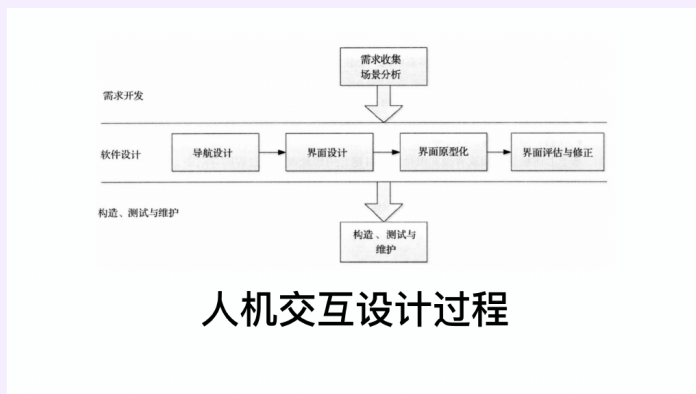
- 语句覆盖：每条语句至少执行一次，强度较弱。
- 判定/分支覆盖：每个判定的真假分支至少执行一次。
- 条件覆盖：每个基本条件的真假至少出现。
- 路径覆盖：覆盖独立路径，强度高但成本大，适合关键复杂代码。

“基于用例的软件测试”可理解为系统测试/验收测试的来源：用例正常流程和扩展流程可以转化为测试场景。但按答疑，不需要额外准备测试图。

七、人机交互：会用原则解释界面好坏

老师答疑说人机交互原则不要求背得很死，只要能指出哪里好/不好并说出合理理由即可。复习时重点掌握以用户为中心、反馈、错误恢复、一致性、减少记忆负担、可用性评估。

人机交互设计过程



人机交互设计贯穿需求开发与软件设计全过程，而非只是最后一步。

39

课件截图：03-04-人机交互设计.pdf 第 39 页，人机交互设计过程。

Golden Rules（黄金原则）

三条核心原则

#	原则	英文	含义
1	让用户掌控系统	Place the user in control	用户能随时撤销、控制交互节奏
2	减少用户记忆负担	Reduce the user's memory load	系统应替用户记忆信息
3	保持一致性	Be consistent	相似操作有相似结果，减少学习成本

43

课件截图：03-04-人机交互设计.pdf 第 43 页，Golden Rules。

Nielsen 10 条可用性启发原则（1994）

业界最广泛使用的可用性评估框架，也是启发式评估的核心检查单：

#	原则	要点
①	系统状态可见	始终告知用户当前状态（进度条、加载提示）
②	与现实匹配	用用户熟悉的词语和概念，而非系统术语
③	用户可控	支持撤销（Undo）、重做（Redo）和随时退出
④	一致性与标准	相似操作应有相似结果，遵循平台规范
⑤	防止错误	设计上预防错误优先于错误后的提示

#	原则	要点
⑥	识别优于回忆	让选项可见，减少用户记忆负担
⑦	灵活高效	为专家提供快捷方式（热键、宏命令）
⑧	简约设计	不包含无关信息，每个额外信息都稀释注意力
⑨	帮助恢复错误	错误信息需明确指出问题并提供解决方案
⑩	帮助与文档	文档易于搜索，以任务为导向，不冗长

与 Pressman 3 条黄金原则的关系：①③对应"用户掌控"，⑥对应"减少记忆"，④对应"保持一致"，Nielsen 在细节上更全面。

44

课件截图：03-04-人机交互设计.pdf 第 44 页，Nielsen 10 条可用性启发原则。

原则	答题表达
用户掌控	让用户能撤销、取消、返回，避免被系统流程强迫。
减少记忆负担	选项可见、信息就近展示，不要求用户记住前一步内容。
一致性	同类操作、术语、布局保持一致，降低学习成本。
及时反馈	操作后系统给出可理解反馈，长任务显示进度。
错误预防与恢复	优先防止错误，错误发生后给出清晰原因和修复路径。
可访问性	考虑可感知、可操作、可理解、鲁棒，例如颜色对比、键盘操作。

八、考前速查：该会画哪些图

图/表	阶段	必须抓住的点
用例图	需求/用户需求	参与者、用例、系统边界、include/extend/generalization。用例是业务目标，不是 CRUD 步骤。
用例描述	需求/用户需求	10 字段；正常流程 + 扩展流程；前置条件不是正常流程步骤。
系统顺序图	需求/系统级需求	Actor 与黑盒 System 的交互，从上到下画消息。
概念类图	需求分析	只含概念类、属性、关联、多重性，不含方法。
包图	体系结构	展示/逻辑/数据层，接口层，PO/VO，依赖方向。可用包表示模块。
4+1 视图	体系结构	逻辑/开发/进程/物理/场景，场景用于验证。
详细顺序图	详细设计	对象/类之间协作，体现 Controller、Service、Entity、DAO 等职责。
状态图	需求/详细设计	先明确主体，再画状态、事件、守卫、动作；能映射为 switch/if 或 State 模式。
流程图/伪代码	详细设计/构造	表达方法内部控制流程，注意复杂度和可测试性。

不重点/不考倾向

- AFSE / 外部 coding 部分不考；软件工程历史相对不考。
- 团队管理概念不会作为重点；即使出现，也偏简答常识。
- 包的原则今年明确不考，但包图、接口、依赖方向仍要会。
- Java 注解、Spring Boot 配置等具体技术细节不考。
- 需求规格说明书完整书写格式不会让考试现场细写，但“系统交互细节要被表达清楚”的思想仍重要。

九、考场答题框架

大题通用框架

1. 读题后先定系统业务目标：解决什么现实问题，业务指标如何度量。
2. 列核心参与者和核心用例：每个用例对应用户可感知的业务目标。
3. 把一个核心用例展开：正常流程、扩展流程、系统顺序图、概念类图。
4. 选架构：用功能域、质量属性和项目约束说明为什么分层/MVC合适。
5. 画包图：分层 + 功能域 + 接口 + PO/VO + 依赖方向。
6. 做详细设计：按 GRASP 分配职责，说明 Controller、Information Expert、Creator 的选择。
7. 分析代码/原则：指出违反的 SOLID/封装/耦合问题，给出重构方案。
8. 写测试：正常、边界、异常；黑盒/白盒依据；系统测试可从用例场景来。
9. 做人机交互评价：用反馈、一致性、错误恢复、减少记忆负担等原则解释。

简答题可能的高频问法

问法	短答抓手
为什么先明确业务目标？	业务目标定义系统建设原因、范围和可度量成功标准，是后续需求分析和验收的追溯源头。
系统顺序图和详细顺序图区别？	前者把系统当黑盒，画 Actor 与 System；后者展开系统内部对象/类协作。
概念类图为什么没有方法？	它是分析阶段的问题域模型，关注领域概念、属性和关系，不描述软件实现职责。
PO 与 VO 区别？	PO 对应持久化/数据库，VO 面向展示层，只传需要展示的数据，隔离变化和减少耦合。
状态图最重要的前提？	先明确主体，再识别状态、转换事件、守卫条件和动作。
如何处理 flag/switch 控制耦合？	用多态、策略模式或拆分职责，把变化封装到具体实现中。
HCI 原则怎么答？	不死背名称，指出界面问题并用用户掌控、反馈、一致性、错误恢复等解释。

最后一条：题目若给同一个系统，不要每问都重新发明系统。保持核心用例、概念类、架构层次、接口对象前后一致，这会让答案显得非常“软件工程”。